회로와 제약조건

halo2에서 계산은 산술 회로(Arithmetic Circuit)로 표현되고, 이 회로의 정확성은 제약조건(Constraint) 시스템을 통해 보장됩니다. 이 문서에서는 회로의 구조와 제약조건 시스템의 동작 원리를 설명합니다.

산술 회로 (Arithmetic Circuit)

회로란?

산술 회로는 수학적 계산을 그래프 형태로 표현한 것입니다. 각 노드는 연산(덧셈, 곱셈)을 나타내고, 엣지는 값의 흐름을 나타냅니다.

```
예시: (a + b) * c 계산



입력: a,  b, c

          |  |  |

          v  v  |

         [+]----+   <- 덧셈 게이트

       |      |

       v      v

      [*]-----   <- 곱셈 게이트

       |

       v
```

```
출력: (a+b)*c
```

halo2에서의 회로 표현

halo2는 회로를 **표 형태(Table)**로 표현합니다. 각 행은 하나의 계산 단계를, 각 열은 값의 종류를 나타냅니다.

```
// 간단한 덧셈 회로 예제
// a + b = c를 검증하는 회로

use halo2_proofs::{
circuit::{Layouter, SimpleFloorPlanner, Value},
plonk::{Advice, Circuit, Column, ConstraintSystem, Error, Instance, Selector},
poly::Rotation,
};

#[derive(Debug, Clone)]
struct AdditionConfig {
advice: [Column<Advice>; 3], // a, b, c 열
selector: Selector, // 게이트 활성화 선택자
instance: Column<Instance>, // 공개 입력
}

#[derive(Debug, Default)]
struct AdditionCircuit {
```

```
a: Value<u64>,

b: Value<u64>,

}
```

제약조건 시스템

제약조건의 역할

제약조건은 회로가 올바르게 계산되었는지 검증하는 규칙입니다. 모든 제약조건이 만족되어야만 증명이 유효합니다.

Gate (게이트)

Gate는 하나 이상의 제약조건을 그룹화한 것입니다. 각 gate는 특정 연산을 정의합니다.

```
impl Circuit<Fr> for AdditionCircuit {

type Config = AdditionConfig;

type FloorPlanner = SimpleFloorPlanner;



fn  configure(meta: &mut ConstraintSystem<Fr>) -> Self::Config {

let  advice = [

meta.advice_column(),

meta.advice_column(),

meta.advice_column(),

];

let  instance = meta.instance_column();
```

```
let selector = meta.selector();

// 덧셈 게이트 정의: a + b = c
meta.create_gate("addition", |meta| {
let s = meta.query_selector(selector);
let a = meta.query_advice(advice[0], Rotation::cur());
let b = meta.query_advice(advice[1], Rotation::cur());
let c = meta.query_advice(advice[2], Rotation::cur());

// 제약조건: s * (a + b - c) = 0
vec![s * (a + b - c)]
});

AdditionConfig {
advice,
selector,
instance,
}
}

fn synthesize(
&self,
config: Self::Config,
```

```
mut  layouter: impl Layouter<Fr>
) -> Result<(), Error> {
layouter.assign_region(
|| "addition",
|mut  region| {
// 선택자 활성화
config.selector.enable(&mut  region, 0)?;


// 값 할당
let  a_cell = region.assign_advice(
|| "a",
config.advice[0],
0,
|| self.a.map(|v| Fr::from(v)),
)?;


let  b_cell = region.assign_advice(
|| "b",
config.advice[1],
0,
|| self.b.map(|v| Fr::from(v)),
)?;
```

```
let c_value = self.a.zip(self.b).map(|(a, b)| Fr::from(a + b));
let c_cell = region.assign_advice(
|| "c",
config.advice[2],
0,
|| c_value,
)?;


Ok(())
},
)
}
}
```

컬럼 타입

halo2는 세 가지 컬럼 타입을 제공합니다:

1. Advice 컬럼

- **용도**: 증명자만 알고 있는 비밀 값 (witness)
- **특징**: 증명 생성 시에만 값이 할당됨

```
// Advice 컬럼 사용 예제
let advice_column = meta.advice_column();
```

```
// 값 할당 (증명 생성 시)
region.assign_advice(
|| "secret value",
advice_column,
row,
|| Value::known(Fr::from(42)),
)?;
```

2. Instance 컬럼

- **용도**: 공개적으로 알려진 값 (public input)
- **특징**: 증명자와 검증자 모두 알고 있음

```
// Instance 컬럼 사용 예제
let  instance_column = meta.instance_column();


// 공개 입력 설정
let  public_input = vec![Fr::from(100)];
```

3. Fixed 컬럼

- **용도**: 회로 설계 시 고정된 상수 값
- **특징**: 모든 증명에서 동일한 값

```
// Fixed 컬럼 사용 예제
```

```
let fixed_column = meta.fixed_column();


// 고정 값 할당 (회로 설계 시)
layouter.assign_region(
|| "constants",
|mut region| {
region.assign_fixed(
|| "constant",
fixed_column,
0,
|| Value::known(Fr::from(7)), // 항상 7
)
},
)?;
```

복잡한 제약조건 예제

곱셈 게이트

```
// 곱셈 게이트: a * b = c
meta.create_gate("multiplication", |meta| {
let s = meta.query_selector(selector);
let a = meta.query_advice(advice[0], Rotation::cur());
```

```
let b = meta.query_advice(advice[1], Rotation::cur());
let c = meta.query_advice(advice[2], Rotation::cur());

// 제약조건: s * (a * b - c) = 0
vec![s * (a * b - c)]
});
```

조건부 제약조건

```
// 조건부 제약조건: if condition then a = b else a = c
meta.create_gate("conditional", |meta| {
let s = meta.query_selector(selector);
let condition = meta.query_advice(advice[0], Rotation::cur());
let a = meta.query_advice(advice[1], Rotation::cur());
let b = meta.query_advice(advice[2], Rotation::cur());
let c = meta.query_advice(advice[3], Rotation::cur());

// condition은 0 또는 1이어야 함
let condition_bool = s.clone() * condition.clone() * (Expression::Constant(Fr::
// 조건부 로직: condition * (a - b) + (1 - condition) * (a - c) = 0
let conditional_constraint = s * (
condition.clone() * (a.clone() - b) +
(Expression::Constant(Fr::one()) - condition) * (a - c)
```

```
);

vec![condition_bool, conditional_constraint]
});
```

범위 제약조건

```
// 값이 특정 범위 내에 있는지 확인 (예: 0 ≤ value < 16)
meta.create_gate("range_check", |meta| {
let s = meta.query_selector(selector);
let value = meta.query_advice(advice[0], Rotation::cur());
// value * (value - 1) * (value - 2) * ... * (value - 15) = 0
let mut constraint = value.clone();
for i in 1..16 {
constraint = constraint * (value.clone() - Expression::Constant(Fr::from(i)));
}
vec![s * constraint]
});
```

제약조건 시스템의 동작 원리

1. 제약조건 수집

회로 설계 시 모든 제약조건이 수집됩니다:

```
// 제약조건 수집 과정 (의사코드)
fn collect_constraints(circuit: &Circuit) -> Vec<Constraint> {
let mut constraints = Vec::new();
// 각 게이트에서 제약조건 추출
for gate in circuit.gates() {
constraints.extend(gate.constraints());
}
// 등식 제약조건 추가
constraints.extend(circuit.equality_constraints());
// Lookup 제약조건 추가
constraints.extend(circuit.lookup_constraints());
constraints
}
```

2. 제약조건 평가

증명 생성 시 모든 제약조건이 0인지 확인합니다:

```
// 제약조건 평가 (의사코드)
fn evaluate_constraints(
constraints: &[Constraint],
witness: &Witness
) -> Result<(), ConstraintError> {
```

```
for (i, constraint) in  constraints.iter().enumerate() {
let  value = constraint.evaluate(witness);
if  value != Fr::zero() {
return Err(ConstraintError::Violation {
constraint_index: i,
actual_value: value
});
}
}
Ok(())
}
```

3. 다항식 변환

제약조건들은 다항식으로 변환되어 commitment됩니다:

```
// 제약조건을 다항식으로 변환 (의사코드)
fn  constraints_to_polynomials(
constraints: &[Constraint],
domain: &Domain
) -> Vec<Polynomial<Fr>> {
constraints
.iter()
.map(|constraint| {
```

```
// 제약조건을 다항식으로 보간
let evaluations: Vec<Fr> = domain
.elements()
.map(|point| constraint.evaluate_at(point))
.collect();
domain.ifft(&evaluations)
})
.collect()
}
```

최적화 기법

1. 선택자 최적화

불필요한 선택자 사용을 줄여 효율성을 높입니다:

```
// 비효율적: 각 게이트마다 별도 선택자
let add_selector = meta.selector();
let mul_selector = meta.selector();

// 효율적: 하나의 선택자로 여러 게이트 제어
let combined_selector = meta.selector();
meta.create_gate("combined", |meta| {
let s = meta.query_selector(combined_selector);
```

```
let  op_type = meta.query_advice(op_type_column, Rotation::cur());

// op_type에 따라 다른 연산 수행

vec![

s.clone() * op_type.clone() * (a + b - c), // 덧셈

s * (Expression::Constant(Fr::one()) - op_type) * (a * b - c), // 곱셈

]

});
```

2. 제약조건 병합

관련된 제약조건들을 하나로 병합합니다:

```
// 비효율적: 별도의 제약조건들

vec![

s.clone() * (a - b),

s.clone() * (c - d),

s * (e - f),

]


// 효율적: 병합된 제약조건

vec![

s * ((a - b) + alpha * (c - d) + alpha.square() * (e - f))

]
```

디버깅 팁

1. 제약조건 위반 찾기

```
// 제약조건 위반 디버깅
fn debug_constraint_violation(circuit: &Circuit, witness: &Witness) {
for (i, constraint) in circuit.constraints().iter().enumerate() {
let value = constraint.evaluate(witness);
if value != Fr::zero() {
println!("제약조건 {} 위반: 값 = {:?}", i, value);
println!("제약조건 내용: {:?}", constraint);
}
}
}
```

2. 값 추적

```
// 중간 값들을 추적하여 디버깅
region.assign_advice(
|| "debug: intermediate value",
debug_column,
row,
|| {
let val = some_computation();
```

```
println!("중간 값: {:?}", val);

Value::known(val)

},

)?;
```

다음 단계

회로와 제약조건의 기본 개념을 이해했다면:

- 증명과 검증: 제약조건이 만족된 회로로부터 어떻게 증명을 생성하는지 학습
- Circuit 설계: 실제 회로 구현 방법 학습
- 제약조건 시스템: 더 복잡한 제약조건 작성법 학습

참고 자료

- PLONK 논문 - 제약조건 시스템의 이론적 배경
- halo2 Circuit trait 문서
- 산술 회로 기초